

Unit-V Transaction Concept

Transaction: Transaction is an action or series of actions that are being performed by single user or an application program.

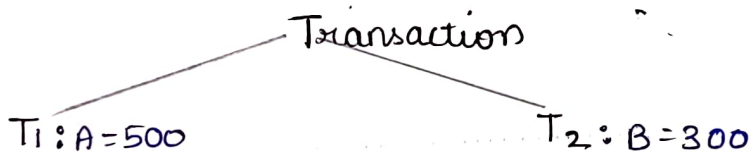
⇒ A transaction can be defined as logical unit of work on the database. i.e. independently executed.

Example:-

Take two accounts A, B and make a transaction of ₹100/- from A to B.

initially account A = 500

account B = 300



Read balance (A) // 500

$A = A - 100$

$A = 500 - 100$

write(A)

$A = 400$

Debited from A is
successful

Read balance (B) // 300

$B = B + 100$

$B = 300 + 100$

write(B)

$B = 400$

credited ^{to} from B is
successful

Transaction operations: There are mainly two types of operations

1. READ(x)

2. WRITE(z)

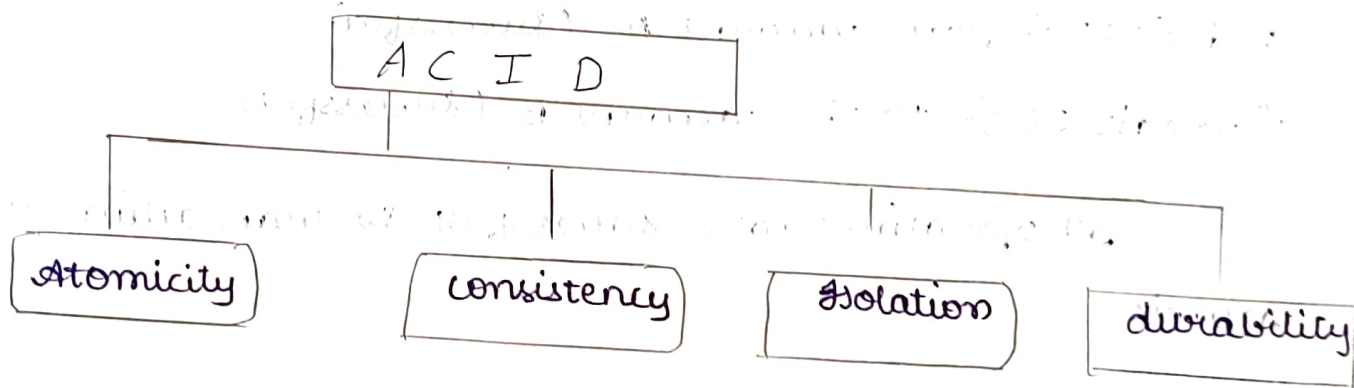
Read operation is used to

→ READ(x): Read the data from x

→ WRITE(x): write operation is used to write the data (insert, delete, update) to account x

* Transaction properties: A transaction must follow the following properties. These properties are called ACID Properties.

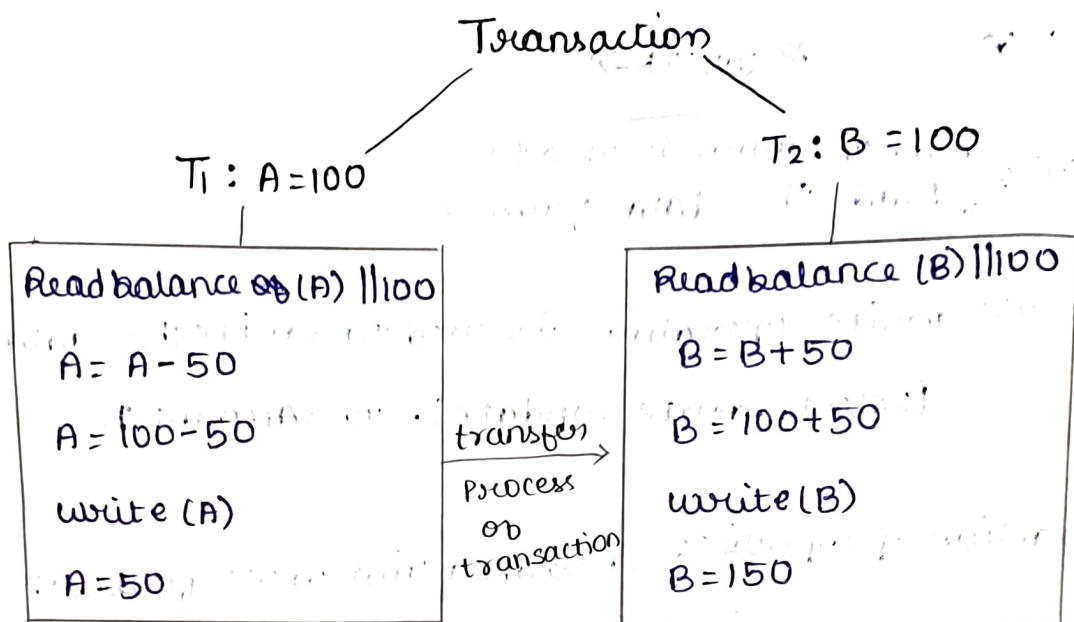
These properties are used to maintain the integrity, consistency, of the data even changes are done.



⇒ Atomicity: 1) This property states that transaction must be treated as atomic i.e. either all of its operations are executed or not.

2) If any operation is performed on data, it should be executed completely (or) it should not be executed at all. (It should not start execution)

Example:-



Debited from A is
successful

Credited to B is
successful

All operations are successful.

This transaction having 2 operations

i. debit ₹50/- from account A (successful)

ii. credit ₹50/- to the account B (successful)

All operations are successful so transaction is
atomic

⇒ consistency:- consistency property states that transaction remains in an consistent state after any transaction. i.e. if a change i.e. if a database is changed before the transaction that should remain constant after execution of whole transaction.

consistency :- database must be in consistent state before and after a transaction.

atomicity :- atomicity means either whole transaction executes or NULL

example :- Take accounts A, B, C transfer the amount from account A to account B & C in sequence so the transaction having two operations

i. debit from account A

ii. credit to account B

debit from account A

credit to account C

Initially account A amount = ₹ 300/-

Initially account B amount = ₹ 100/-

Initially account C amount = ₹ 50/-

Transaction

$T_1: A = 300$

$T_2: B = 100$

$T_3: C = 50$

Read (A)

$A = A - 50$

$A = 300 - 50$

Write (A)

$A = 250$

After transfer 50

Account C

Read (A)

$A = 250 - 50$

Write (A)

$A = 200$

Read (B)

$B = B + 50$

$B = 100 + 50$

Write (B)

$B = 150$

Read (C)

$C = C + 50$

$C = 50 + 50$

Write (C)

$C = 100$

before transaction account A = 300

B = 100

C = 50

total amount = 450

After transaction an account A = 200

B = 150

C = 100

450

i. Read the account balance A = ₹300/- (Before transaction)

transfer ₹50/- to acc. B

Now acc. A = ₹250/- (After transaction (A → B))

ii. Read the acc. balance B = ₹¹⁰⁰~~50~~/- (Before transaction)

After credited ₹50/- to B

Account B = ₹150/- (After transaction)

iii. Read the acc. balance A = ₹250/- (Before transfer to C)

transfer ₹50/- to C

Now acc A = ₹200/- (After all transactions)

iv. Read acc C = ₹50/- (Before transaction)

After credited ₹50/- to C

Now acc C = ₹100/- (after transaction)

∴ After transaction A = ₹200/-

B = ₹150/-

C = ₹100/-

Before transaction = After transaction.

Isolation: This property states that multiple transactions can occur concurrently without disturbing one another. By using this property, data inconsistency will be avoided. ~~when~~

when two or more transactions occur simultaneously, the consistency should be maintained. The change in one transaction will not be effected in the another transaction until the first transaction committed.

Transaction:

$T_1: A = 100$

Read(A) // 100

$A = A - 20$

$A = 100 - 20$

write(A)

$A = 80$

$T_2: B = 50$

Read(B) // 50

$B = B + 20$

$B = 50 + 20$

write(B)

$B = 70$

$T_3: C = 70$

Read(C) // 70

$C = C + 20$

$C = 70 + 20$

write(C)

$C = 90$

Read(A) // 80

$A = A - 20$

$A = 80 - 20$

write(A)

$A = 60$

i, Read the account balance $A = ₹100/-$ (Before transaction)
transfer the amount to account B i.e ₹20/-

ii Now, Read the account $A = ₹80/-$ (After transaction $(A \rightarrow B)$)

iii Read the account Balance of $B = ₹50/-$ (Before transaction)

iv Now, the amount in account B is ₹70/-

v, Read The account balance $A = ₹80/-$ (Before transaction to
transfer the amount to account C i.e ₹20/-

vi Now, Read the account Balance of $C = ₹70/-$ (Before transaction)

vii, now the amount in account C is ₹90/- (After transaction
-on)

$\therefore$ After transaction $A = ₹60/-$

$B = ₹70/-$

$C = ₹90/-$

₹220/-

$\therefore$ Before transaction = After transaction (i.e ₹220/-)

$\Rightarrow$ Durability: This property ensures that once the transaction has been completed and executed the updates and modifications to the database are stored in and return to the disk and it is present in the system even failure occurs.

i.e even if the system failure occurs due to the software or hardware database must be present or survive.

Isolation :- Isolation means execution of a transaction should not interfere with execution of another transaction.

Durability :- Durability means data must not be lost even if system fails.

* Transaction states :- Transaction must have the following states

i. Active / Start

ii. Partially committed

iii. failed

iv. Aborted

v. committed

vi. Terminated.

⇒ Active / start state :- It is a initial state

* When the transaction start the execution we can say that transaction is active.

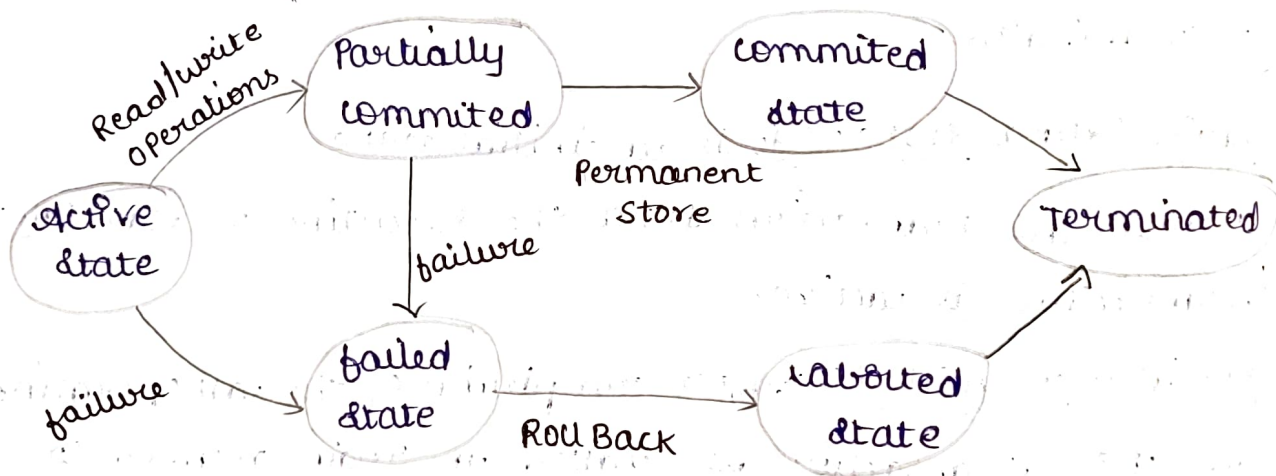
⇒ Partially committed :- After completion of the all operations with the transaction is called in main memory (or) local Buffer. This state of transaction is called partially committed state. But it is not saved in database

⇒ Committed state :- If all the operations of transactions are completed successfully and all the changes are called permanently in the database system.

⇒ Failed state:- when any operations of transaction fails then the state of transaction is called failed state.

⇒ Aborted state:- After the fail state transaction goes to aborted state. Here if the changes were done in the main memory (or) local buffer those changes are deleted or transactions were rolled back.

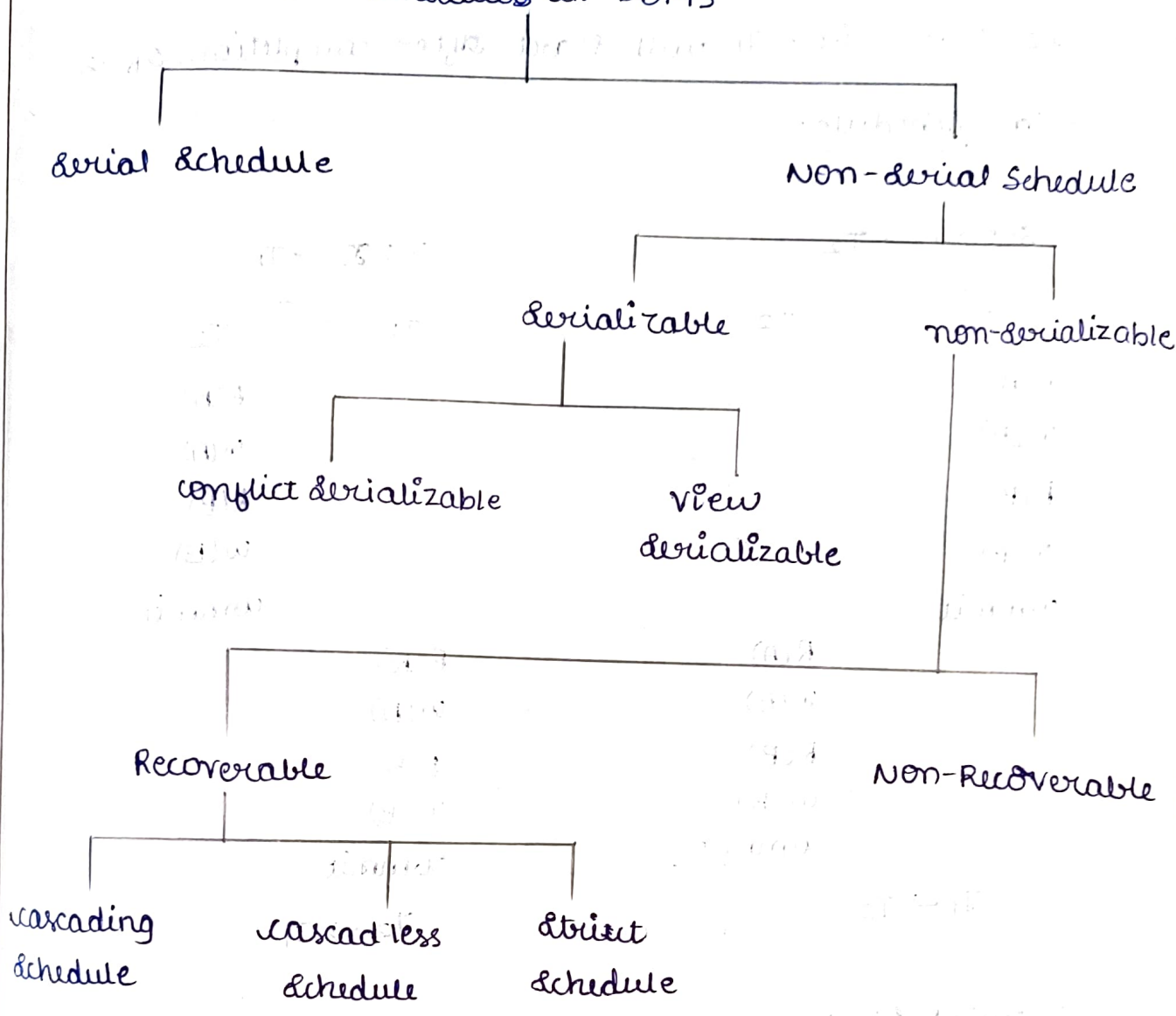
⇒ Terminated state:- If the transaction is successfully committed then the transaction is terminated. If there is not any roll back then the transaction is successfully terminated.



"Transaction state"

* Serializability:-

Scheduled in DBMS



Schedule:- The sequence of operations in a transactions is known as schedule

There are mainly two types of schedule

- i. Serial Schedule
- ii. Non-serial schedule

Serial schedule:- In serial schedule one transactions will start execution of after completion of the all operations of another transactions.

Example:- let T_1, T_2 be two transactions.

$T_1 \rightarrow T_2$ i.e; T_2 will start after completion of T_1

$T_2 \rightarrow T_1$ i.e; T_1 will start after completion of T_2

Serial schedule:-

$S_1: T_1 \rightarrow T_2$

T_1	T_2
$R(A)$	
$w(A)$	
$R(B)$	
$w(B)$	
commit	
	$R(A)$
	$w(A)$
	$R(B)$
	$w(B)$
	commit

$T_1 \rightarrow T_2$

$S_2: T_2 \rightarrow T_1$

T_1	T_2
	$R(A)$
	$w(A)$
	$R(B)$
	$w(B)$
	commit
$R(A)$	
$w(A)$	
$R(B)$	
$w(B)$	
commit	

$T_2 \rightarrow T_1$

$\therefore$ serial schedule

Non-serial schedule:- In non-serial schedule one transaction starts its execution before completion of the another transaction i.e; concurrent execution of the transaction are allowed in non-serial schedule.

Properties of non-serial schedule:- (Not always)

1. Recoverable
2. Strict
3. Cascade Cascade less

Serial Schedule Explanation :-

Initially; $A = 100$ $B = 100$ in $T_1(S_1)$

In T_1 debit ₹50/- from A and credit to B

∴ At the end of T_1 $A = 50$, $B = 150$

$$\therefore A + B = 200$$

⇒ Initially in T_2 , (After completion T_1)

$$A = 50, B = 150$$

In T_2 , debit ₹20/- from A and credit to B

∴ At the end of T_2 $A = 30$, $B = 170$

$$\therefore A + B = 200$$

∴ There is no data inconsistency

∴ It is a serial schedule

Types of Non-serial Schedule :-

There are two types

1. Serializable Schedule
2. Non-serializable Schedule

Non-serial schedule:-

S3

T ₁	T ₂
R(A) 50 W(A) 30 ²⁰	R(A) 30 ¹⁰ W(A) 20
R(B) 50 ⁺²⁰ W(B) 70	R(B) 70 ⁺¹⁰ W(B) 80
commit	commit

S4

T ₁	T ₂
R(A) 50 W(A) 30	R(A) 50 W(A) 40
R(B) 50 W(B) 70	R(B) 70 W(B) 80
commit	commit

Non-serial without
conflict

Non-serial with conflict

In S4 schedule, T₂ performed R(A) before completion of T₁(W(A)). So, it is conflict Non-serializable schedule.

Non-serial schedule explanation:-

For non-serial schedule - S3:

Initially in T₁(S3)

$$A = 100$$

In, T₁ debit ₹ 50/- from A

Initially in T₂ (S3) [After completion of T₁(W(A))]

$$A = ₹ 50/-$$

In T₂ debit ₹ 20/- from A then A = ₹ 30/-

Initially in $T_1(S_3)$

$$B = 100$$

Now, credit ₹50/- to account B then $B = ₹150/-$

Initially in $T_2(S_3)$ [After completion of $T_1(w(B))$]

$$B = ₹150/-$$

Now, credit ₹20/- to account B

$$\therefore B = ₹170/-$$

At the end of $T_2(S_3)$ $A = 30$, $B = 170$

$$\therefore A + B = 200$$

For non-serial schedule - S_4

* Initially, $A = 100$ in $T_1(S_4)$.

In T_1 debit ₹50/- from A

* Initially, $A = 100$ in $T_2(S_4)$ [Before completion of $T_1(w(A))$]

Now, debit ₹20/- from A

* After completion of $T_2(w(A))$ we are performing

$T_1(w(A))$ then, $A = 80$.

* Initially, $B = 100$ in $T_1(S_4)$.

Now, credit ₹50/- to acc. B then $B = ₹150/-$

* Initially in $T_2(S_4)$ [After completion of $T_1(w(B))$]

$$B = ₹150/-$$

Now credit ₹20/- to acc B then $B = ₹170/-$

$\therefore$ At the end of the T_2

we have, $A = ₹80/-$ and $B = ₹170/-$

$$\therefore A+B = ₹250/-$$

But according to transactions we should get $A+B=200$

Because, initially we have $A=100$ & $B=100$

But in su schedule, T_2 performs $R(A)$ before completion $T_1(W(A))$

$\therefore$ It leads to conflict, Non-serializable schedule

* Serial Example:

$$A=100; B=100$$

$$T_1: A \rightarrow B; T_2: A \rightarrow B$$

$$T_1 \rightarrow T_2$$

T_1	T_2
$R(A)=100$	
$A = A - 50$	
$W(A)=50$	
$R(B)=100$	
$B = B + 50$	
$W(B)=150$	
$R(A)+R(B)=100+100$ $=200$	
$W(A)+W(B)=50+150$ $=200$	
	$R(A)=50$
	$W(A)=A-50$
	$W(A)=20$
	$R(B)=150$
	$B = B + 30$
	$W(B)=180$
	$R(A)+R(B)=50+150$
	$W(A)+W(B)=20+180$

$$B = 100$$

Non-serial without conflict table (Refer Back) After that write Serializable schedule then

Serializable schedule :-

Types of serializability

Types of serializability

i.e. if non-serial schedule becomes to serializable

A schedule is called serializable schedule:

Example:-

T ₁	T ₂		T ₁	T ₂
R(A)			R(A)	
	R(B)		w(B)(A)	
w(A)		} swap →		R(B)
	w(B)			w(B)
commit			commit	
	commit			commit

non serial



serial

serializable schedule.

Non-serial schedule without conflict:-

A = 100 ; B = 100.

T_1	T_2
$R(A) = 100$ $A = A + 50$ $W(A) = 50$ $B = 100$ $B = B + 50$ $W(B) = 150$ commit $A + B = 200$	$R(A) = 50$ $A = A - 30$ $W(A) = 20$ $R(B) = 150$ $B = B + 30$ $B = 180$ commit $A + B = 200$

* Types of serializable schedule:- there are two types of serializable schedule.

i. conflict serializability

ii. view serializability

* conflict serializability:-

A schedule is called conflict serializability after swapping of non-conflicting operations then it can transform to conflict serial schedule.

* conflicting pairs:-

Read of A - Read(A) - write(A)

write(A) - write(A)

write(A) - Read(A)

* conflicting operations:-

The two operations become conflicting if all condition satisfy.

i, Both belongs to separate transactions

ii, Both have same data item.

iii, Both contain atleast one write operation.

T ₁	T ₂	
Read(Q)	Read(Q)	Non-conflict
Read(Q)	write(Q)	conflict
write(Q)	read(Q)	conflict
write(Q)	write(Q)	conflict.

* operations are on different data items - NON CONFLICTING

* operations are on same data items - CONFLICTING / -3

NON CONFLICTING -1

(for above table)

Example:- (conflict serializability)

S

Step:- 1

Swap
 $r(A) - T_1$
and
 $r(A) - T_2$

T1	T2
read(A)	
	read(A)
	read(B)
	write(B)
read(B)	
write(A)	

Swap

T1	T2
	read(A)
read(A)	
	read(B)
	write(B)
read(B)	
write(A)	

Swap
 $r(A) - T_1$ &
 $r(B) - T_2$

Swap

Step:- 2

T1	T2
	read(A)
	read(B)
read(A)	
	write(B)
read(B)	
write(A)	

Swap

Step:- 3

T1	T2
	read(A)
	read(B)
	write(B)
read(A)	
read(B)	
write(A)	

Swap $r(A) - T_1$ &

$w(B) - T_2$

$\therefore$ Non-serial S becomes to serial

$\therefore$ S is a conflict serial schedule.

* View serializability :- View serializability is a process to find out that a given schedule is view serializable or not.

Equivalent

If a schedule is view equivalent to its serial schedule then it is view serializable.

Equivalent

* View equivalent :- Two schedules S_1 - A non-serial schedule

S_2 - A serial schedule

are said to be view equivalent if they satisfy the

following conditions:

I, Initial read

II, Final write

III, update read

Each data item are same/
on same data item.

1. Initial read

T1	T2
Read(A)	
	write(A)

T1	T2
Read(A)	
	write(A)

2. Update read

T1	T2	T3
write(A)		
	write(A)	
		Read(A)

T1	T2	T3
	write(A)	
write(A)		
		Read(A)

3. Final write

T1	T2	T3
write(A)		
	Read(A)	
		write(A)

T1	T2	T3
	Read(A)	
write(A)		
		write(A)

Example :- view serializability.

S1	
T1	T2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

Initial read(A) →
 update read(A) →
 Final write(A) →
 Initial read(B) →
 update read(B) →
 Final write(B) →

S2	
T1	T2
read(A)	initial(A)
write(A)	
read(B)	initial(B)
write(B)	
update(A)	read(A)
final(A)	write(A)
update(B)	read(B)
final(B)	write(B)

initial R(A) - T1 initial R(B) - T1 initial R(A) - T1 initial R(B) - T1
 update R(A) - T2 update R(B) - T2 update R(A) - T2 update R(B) - T2
 Final W(A) - T2 Final W(B) - T2 Final W(A) - T2 Final W(B) - T2

∴ S1 and S2 are view equivalent.

∴ S1 is view serializable schedule.

* Testing for serializability :- By using the precedence graph we can test the conflict serializability.

* If the graph contains cycle the schedule is not a conflict serializable schedule.

* If the graph not contains cycle the schedule is called conflict serializable schedule.

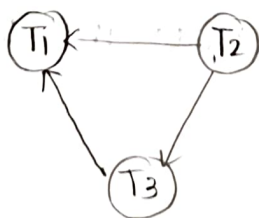
T_1	T_2	T_3
$R(x)$		
		$R(y)$
		$R(z)$
	$R(y)$	
	$R(z)$	
		$w(y)$
	$w(z)$	
$R(z)$		
$w(x)$		
$w(z)$		

A precedence graph is known as 'serialization graph' or conflict graph. It is a directed graph.

Procedure :-

- 1) No. of transactions are taken as no. of vertices.
- 2) No. of edges are drawn by using initial read - final write pairs of different transactions.

Let T_1, T_2, T_3 are vertices



$T_2 \rightarrow T_1$

$T_2 : R(z) \rightarrow T_1 : w(z)$

$T_2 \rightarrow T_3$

$T_2 : R(y) \rightarrow T_3 : w(y)$

$T_3 \rightarrow T_1$

$T_3 : R(x) \rightarrow T_1 : w(x)$

Example:- $h = R_1(x) R_2(x) W_1(x) R_1(y) W_2(x) W_1(y)$

T_1	T_2
$R_1(x)$	
	$R_2(x)$
$W_1(x)$	
$R_1(y)$	
	$W_2(x)$
$W_1(y)$	



* Recoverability:- A non-serial schedule which is not serializable is called non-serializable schedule.

A non-serializable schedule is two types:-

1) Recoverable schedule

2) Not - Recoverable schedule

⇒ Recoverable schedule:- when a transaction performs a dirty read from an uncommitted transaction. (1st transaction)

⇒ But the commit operation is done only after uncommitted ^{first} transaction is committed or rollback.

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_1(A)$
	$W_1(A)$
Commit	
	Commit

T_1 commits.

// Dirty read

// Dirty delay commit.

→ T_2 commits after T_1 commits.

commits.

- * Here T_2 performs the dirty read from the T_1 .
- * T_2 waits to perform commit until T_1 commit or rollback
- * T_2 will commit after T_1 only, then all the changes are reflects to T_2 . This schedule is called recoverable schedule

⇒ Example:-

T_1	T_2
$R(A) = 100$ $A = A - 50$ $W(A) = 50$	
	$R(A) = 50$ $A = A + 150$ $W(A) = 200$
// $A = 50$ commit	
	commit // $A = 50 + 150$

Initial $R(A) \Rightarrow 100$ (Before commit)
 $W(A) \Rightarrow 50$
 (after commit)

Initial $R(A) \Rightarrow 100$ (Before commit)
 $W(A) \Rightarrow 200$

- * Initially T_1 Reads $A = 100$ and perform write operation $A = A - 50$ but it is not committed in T_1
- * T_2 reads $A = 100$ (from uncommitted T_1) it performs write operation $A = A + 150$. Now $A = 250$ but it is not committed in T_2
- * Now T_1 is committed so change of A in T_1 reflects to T_2 (A) Now $T_2(A) = 50$ so then $A = 200$ ($50 + 150$) now T_2 is committed.
- ∴ It is a recoverable schedule (There is no data inconsistency)

⇒ Irrecoverable Schedule :- A transaction performs a dirty read from an uncommitted transaction (1st transaction)
 ⇒ But the 2nd transaction (T2) performs the commit operation before the 1st transaction is committed or rolled back. This schedule is known as irrecoverable schedule.

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
	commit
commit	

// Dirty read

⇒ T2 commits before T1

Rollback
A = 50

// A = 100
T1

T1	T2
R(A) = 100	
A = A - 50	
W(A) = 50	
	A = 50
	A = A + 150 50 + 150
	A = 200
	commit
Rollback	
commit	

// A = 200

- * Initially T1 reads A = 100 and it performs the write operation $A = A - 50$. Therefore W(A) = 50 at T1 (Before commit)
- * T2 reads A = 50 (dirty read from internal memory). T2 performs write operation $A = A + 150$
 $A = 50 + 150$
 $\therefore W(A) = 200$ (After T2 is committed)
- * After T2 committed T1 is rolled back its write operation.
 $\therefore W(A) = 100$ But the change is not reflected to the T2(A) Because T2 is already committed.
 $\therefore$ Inconsistency of data is occurred (i.e. in place of 250 we got 200)
 $\therefore$ It is an irrecoverable schedule.

* Types of recoverable schedule:-

1) cascading schedule

2) cascades schedule

3) strict schedule.

Recoverable schedule

cascading schedule

cascades schedule

strict schedule

⇒ cascading schedule:- In a schedule, Failure of one transaction causes to failure of several other dependent transactions to rollback (or) abort then that schedule is known as cascading schedule (or) cascading rollback (or) cascading abort.

T ₁	T ₂	T ₃	T ₄
R(A)			
W(A)			
	R(A)		
	W(A)		
		R(A)	
		W(A)	
			R(A)
			W(A)
Failure			

another example on
H-page

Here T₂ transaction depends on T₁

T₃ depends on T₂

T₄ depends on T₃
transaction T₂ to

Example:-

In the schedule, Failure of T₁ causes the rollback

Rollback of T₂ causes the Rollback of T₃

Rollback of T₃ causes the Rollback of T₄

This schedule is called as cascading & Rollback.

Example:-

T ₁	T ₂	T ₃	T ₄
R(A)=10 w(A)=10+5 w(A)=15			
	R(A)=15 w(A)=15+5 w(A)=20		
		R(A)=20 w(A)=20+5 w(A)=25	
			R(A)=25 w(A)=25+5 w(A)=30

⇒ Cascadless Schedule :- In the schedule, the second transaction is not allowed to read the data item, until the first transaction is committed or aborted. Such a schedule is called as cascadless schedule.

T ₁	T ₂	T ₃
R(A)		
w(A)		
commit		
	R(A)	
	w(A)	
	commit	
		R(A)
		w(A)
		commit

T ₁	T ₂	T ₃
R(A)=10		
w(A)=10+5		
w(A)=15		
commit		
	R(A)=15	
	w(A)=15+5	
	w(A)=20	
	commit	
		R(A)=20
		w(A)=20+5
		w(A)=25
		commit

⇒ Strict Schedule :- In a schedule, the second transaction is allowed to read or write data item after first transaction is committed or aborted. Such a schedule is called strict schedule.

* Strict schedule are more strict than cascadless schedules.

* All strict schedules are cascadless schedule.

* But all cascadless schedules are not strict.

Example :-

T ₁	T ₂
R(A)	
w(A)	
Commit	
	R(A)
	w(A)
	Commit

T ₁	T ₂
R(A)=10	
w(A)=10+5	
w(A)=15	
commit	
	R(A)=15
	w(A)=15+5
	w(A)=20
	commit

In the schedule, T₁ performs the read operation R(A)=10 & it performs the write operation.

$$w(A) = 10 + 5 = 15$$

∴ w(A)=15 is committed and T₂ reads the committed data item R(A)=15 and it performs write operation 15+5=20 w(A)=20

∴ There is NO data inconsistency Because T₂ reads after T₁

* Concurrent Execution :- In the transaction database system more than one transactions are executed simultaneously. The process is called concurrent execution.

Advantages :-

- 1) Decrease the waiting time
- 2) Increase the performance
- 3) Increase the resource utilization.

* Concurrency Control Problems :- There are some problems or conflicts occurred when a concurrent execution is performed.

- 1) Lost update problem (WW conflict)
- 2) Dirty read Problem (RW conflict)
- 3) Unrepeatable Read (RR conflict)

* Lost update Problem :- It occurs when 2 transactions in a database access the same data item.

Let T₁, T₂ are 2 transactions. T₁ access the same & item value & update it

Then second record ~~is~~ overwrites the First record.

Example :- 1

T ₁	T ₂
R(A) // 100	
A = A - 50	
	R(A)
	A = A + 100
W(A) // 50	
	W(A)

→ lost update

A = 200

200 .

Example :- 2

T ₁	T ₂
R(X) // 100	
X = X + 10	
	R(X)
	X = X - 20
W(X)	
	W(X)

* T₂ reads, A which is not updated in T₁

* so there is inconsistency occurred

T₂(W(A)) = 200

Instead of W(A) = 150
lost update.

Here in S₂, T₂ reads the x value which is not updated so, there is inconsistency occurred.

T₂(W(X)) = 80

instead of W(X) = 90

* Dirty read Problem :-

T ₁	T ₂
R(A) W(A)	
	R(A) W(A)
Failed X	X

T ₁	T ₂
R(A) = 100 W(A) = A + 50 W(A) = 150	
	R(A) = 150 W(A) = 150 + 50 = 200
Failed Rollback	

In this schedule, the second transaction reads the item from a first transaction which is uncommitted.

T₂ performs the write updation on the data item

Now, T₁ transaction gets failed & rollbacked the transaction before it is committed.

so there is a inconsistency in the T₂ transaction with Dirty

read of T1

* T2 reads the $A = 150$ & performs $W(A) = 150 + 50$ Now,

$$T_2(W(A)) = 200$$

- Inconsistency occurred.

Because T1 is rolled back to $A = 100$ //

* Unrepeatable read:- This problem occurs when one transaction produces two different values for same data item.

T1	T2
R(A)	
	R(A) W(A)
R(A)	

T1	T2
R(A) = 100	
	R(A) = 100 A = A + 100 W(A) = 200
R(A) = 200	

Here transaction T1 reads initially $A = 100$ & at the end of the transaction again it reads $A = 200$

* Implementation of Isolation:-

[Refer in ACID Properties]

* Failure & classification:- DBMS performs several transactions in a schedule & sometimes these transactions are fails. These failures are classified into mainly 3 types.

1. Transaction failure

2. System crash

3. Data transfer failure.

⇒ Transaction Failure:- If a transaction is failed to execute the operation then the transaction is called failure transaction.

Reasons for transaction failure:-

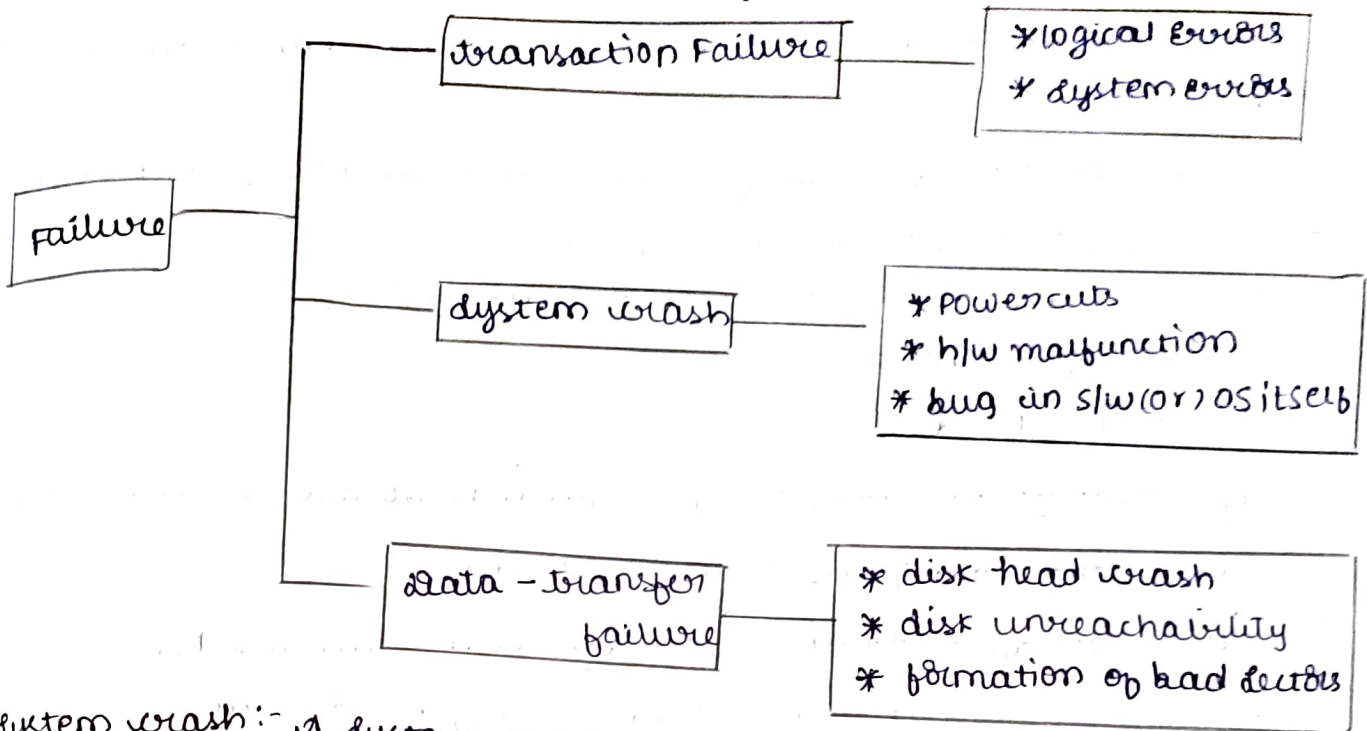
Logical error:- If a logical error occurs because of some error in a code so the transaction will fail.

System error:- If the transaction is successful but upto its end operation even though it fails because of some issue in the system.

Ex:-

If a transaction meets deadlock condition the system automatically terminates them.

Failure classification in DBMS



⇒ System crash:- A system crash occurs usually by the problem of Hardware and software break down.

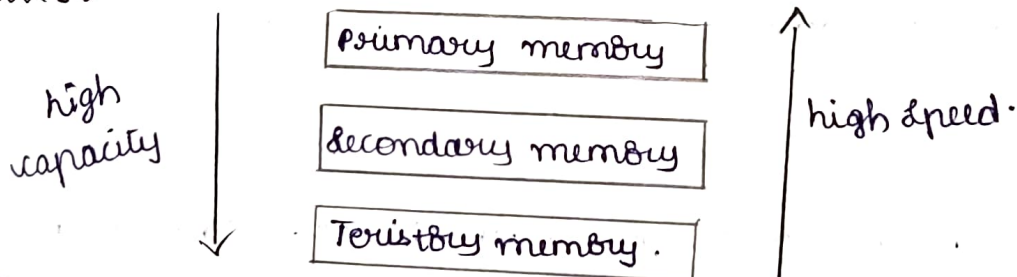
⇒ Operating system errors, powercuts, main memory crash are also leads to system crash.

⇒ Data transfer:- This failure occurs when the information is damaged or crashing which is stored in the disc. Because of crash of disc data transfer will not be possible.

⇒ Disk head crashing formation of bad sectors will leads to disk failure

⇒ To overcome this failure we need to backup the data

* Storage:-



There are three types of storage system in DBMS

Primary storage:- It is a volatile storage.

* It is primary available to access quickly of stored data.

* If the power cut or crash occurs we lost our data.

Example:- 1. main memory

2. cache memory.

⇒ main memory:- main memory handles the each and every

institution of stored data

cache memory:- cache memory is the faster than main memory to access.

Secondary storage:- Secondary storage data used to permanently

⇒ This will not last the data gives

⇒ This is also called as non-volatile storage

Ex:- 1) crash memory

2) magnetic disk

crash memory:- It stores the data in USB so that we can build up it access the data faster than the main memory.

tertiary storage:- Tertiary storage is the storage type is the storage type of External storage or offline storage.

These are used for backup.

⇒ Here the information is store in more than one files so the data will be never lost

Ex:- 1) optical storage

2) Tap storage.

Recovery and automaticity:- Refer acid properties and schedules

Recovery algorithm:- There are 3 phases in the recovery algorithm

1) Analysis phase (or) undo only

2) Redo phase

3) Undo phase

The algorithm used for recovery in DBMS is known as ARIES

ARIES → Algorithm for recovery in DBMS and isolation

Exploring semantics.

A ⇒ Algorithm

S ⇒ semantics

R ⇒ Recovery

I ⇒ Isolation

E ⇒ Exploring

- ⇒ analysis phase:- The recovery subsystem determines the earliest log record. from which next operation must start, it also scans the log forward from the check point record it will take a snap of the system how it looks at the instance or trash
- ⇒ sb only before image is recorded.
- ⇒ Read phase:- only after image is recorded.
- ⇒ It starts at the earliest LSN (log sequence number).
- ⇒ undo phase:- Both before the image & after the image are log
- ⇒ The log is scanned backward and forward and its updates the last transaction and done

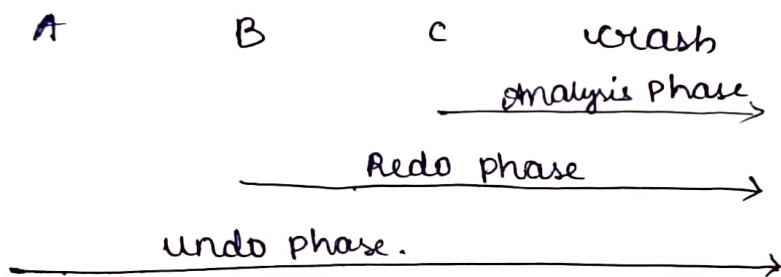


fig:- Recovery phases

• Here A = oldest log record

B = smallest LSN (or) Redo record

C = most recent checkpoint (or) Redo record.